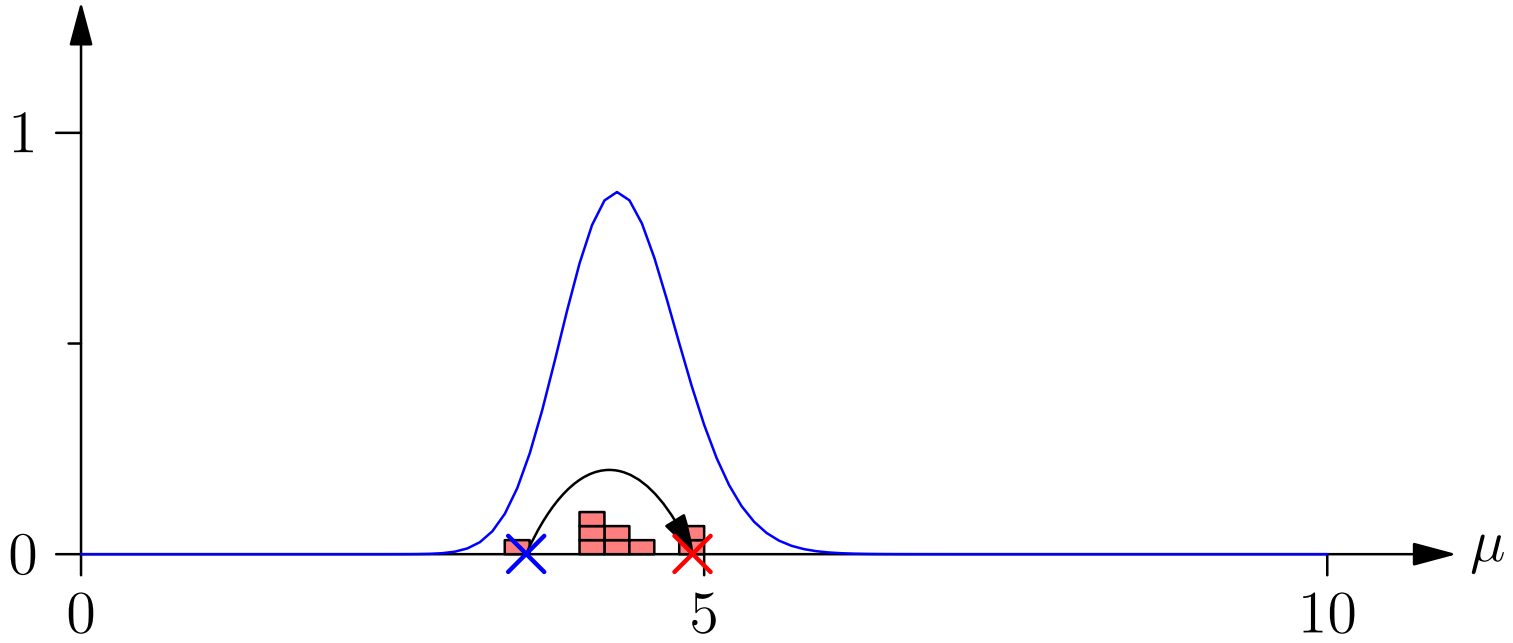
Mathematics of Machine Learning

MCMC

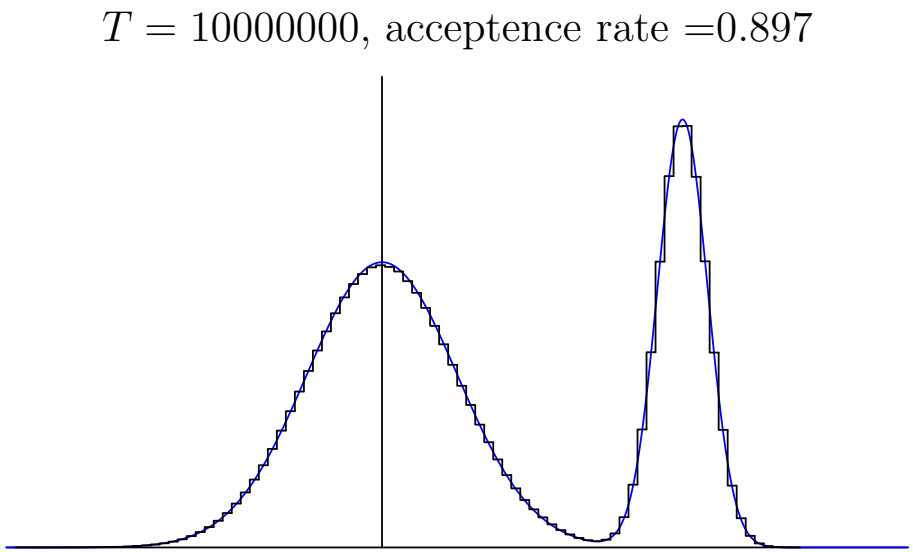
$$\mathcal{D} = \{6, 2, 6, 2, 3, 4, 3, 5, 4, 8, 1, 3, 3, 8, 5, 9, 4, 6, 5, 0\}$$



Monte Carlo methods, MCMC, Metropolis-Hastings, Gibbs sampling, Hamiltonian Monte Carlo, Convergence diagnostics

Outline

1. **Monte Carlo**
2. MCMC
3. Modern MCMC



Bayesian Inference Gets Hard

- With a simple likelihood (normal, binomial, Poisson, multinomial) a conjugate prior gives a posterior of the same form.
- Most real models (e.g. hierarchical models) have no conjugate prior.
- We can still compute the top of Bayes' rule

$$f(\boldsymbol{\theta}|\mathcal{D}) = \frac{f(\mathcal{D}|\boldsymbol{\theta})\, f(\boldsymbol{\theta})}{f(\mathcal{D})}.$$

- But the evidence $f(\mathcal{D})$ is an integral over all $\boldsymbol{\theta}$, which we usually cannot do: this lecture shows how to draw samples from the posterior anyway.

Histograms, Samples and Means

- We could represent our posterior as a histogram, although for multivariate distributions (i.e. when we are modelling more than one variable) a histogram can be unwieldy.
- A sample from the posterior distribution is often sufficient e.g. in a topic model of documents a typical set of topics is what we are after.
- However, when samples vary a lot, often the most useful quantities are expectations, e.g.

$$\mathbb{E}[\boldsymbol{\Theta}] \qquad \mathbb{E}\left[\Theta_i^2\right] - \mathbb{E}[\Theta_i]^2$$

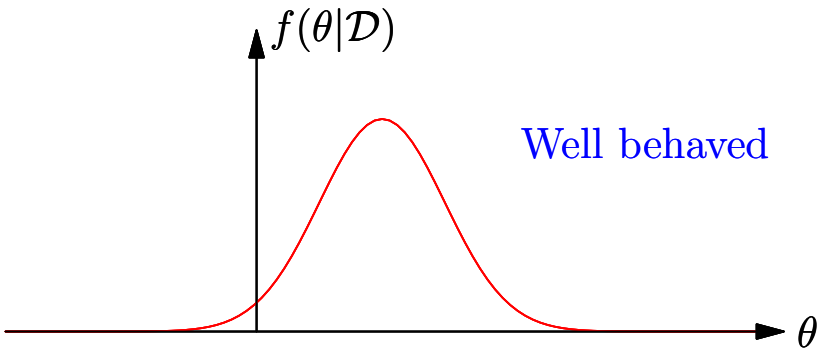
$$\mathbb{E}[\Theta_i\Theta_j] - \mathbb{E}[\Theta_i]\mathbb{E}[\Theta_j] \qquad \mathbb{E}\left[\boldsymbol{\Theta}\boldsymbol{\Theta}^\mathsf{T}\right] - \mathbb{E}[\boldsymbol{\Theta}]\mathbb{E}[\boldsymbol{\Theta}]^\mathsf{T}.$$

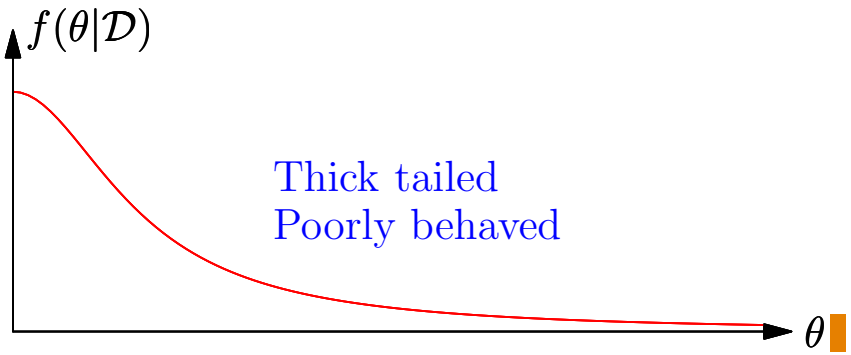
Sample Estimation

- If we can draw independent samples (aka **deviates** or **variates**), $\boldsymbol{\Theta}_i$, from our posterior distribution then we can obtain an estimate of our expectation

$$\mathbb{E}[g(\boldsymbol{\Theta})] \approx \frac{1}{n}\sum_{i=1}^{n} g(\boldsymbol{\Theta}_i)$$

- Provided our posterior distribution is well behaved (the variance of $g(\boldsymbol{\Theta})$ is finite) the error in our estimate will drop off as $1/\sqrt{n}$



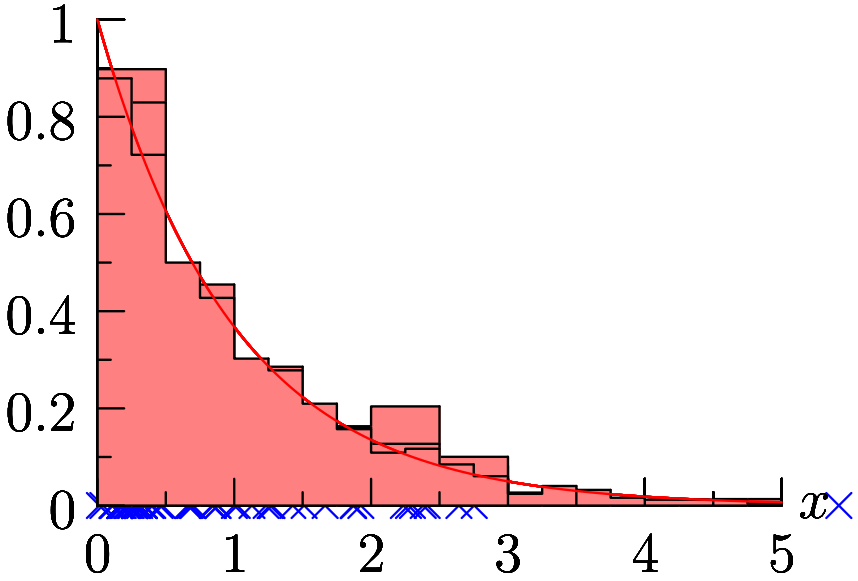


Drawing Random Samples

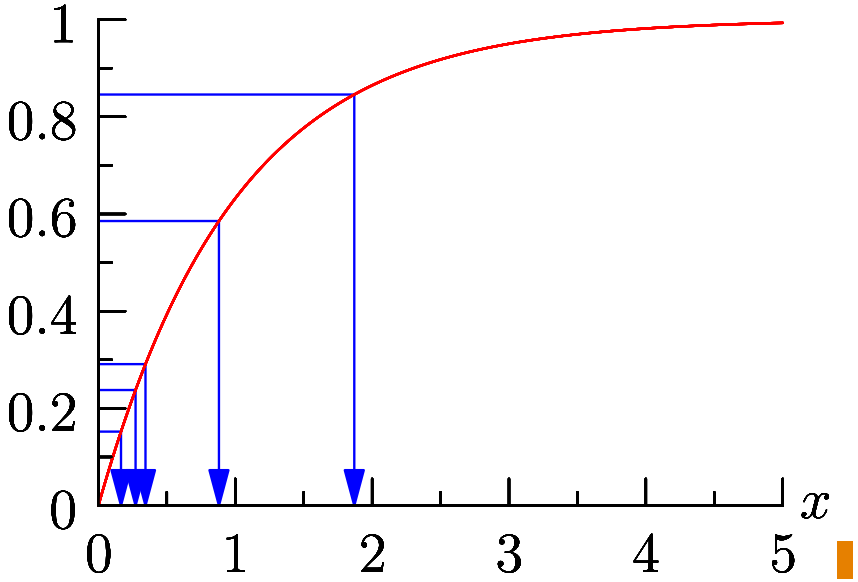
- Computing with (pseudo) random samples drawn from a distribution is known as **Monte Carlo**

- For some very simple distributions we can use the **transformation methods** to transform a uniform distribution

$$\mathrm{Exp}(x|1) = \mathrm{e}^{-x}$$



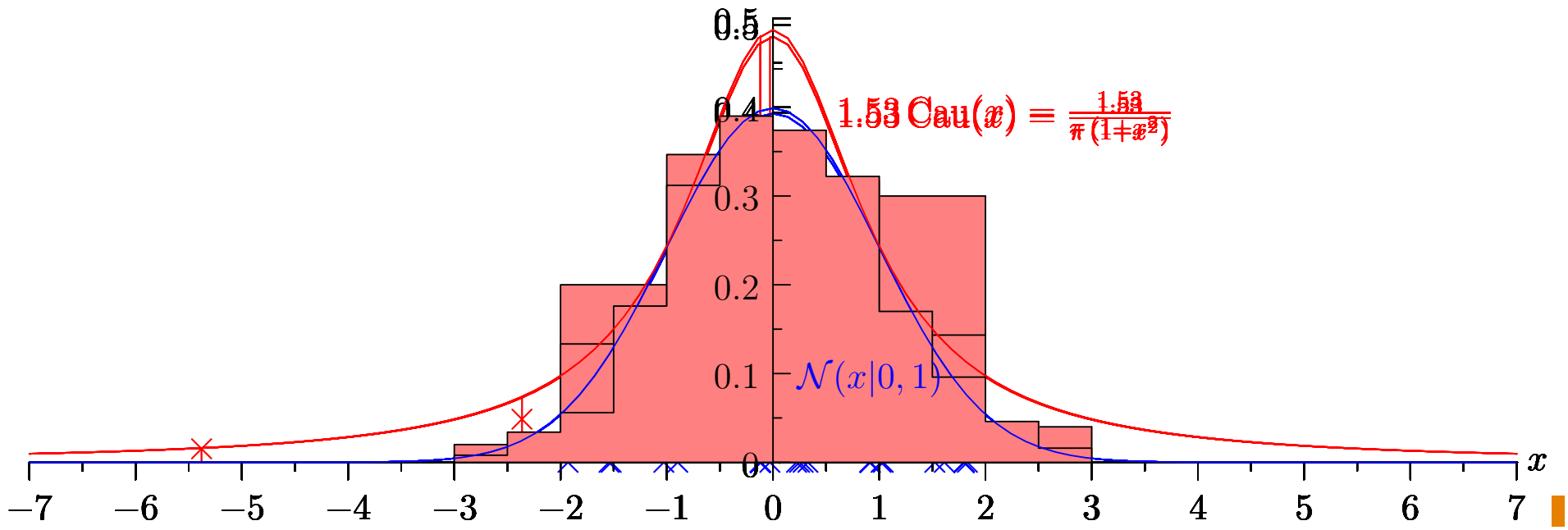
$$F(x|1) = \int_0^x \mathrm{Exp}(y|1)\mathrm{d}y = 1 - \mathrm{e}^{-x}$$



Rejection Method

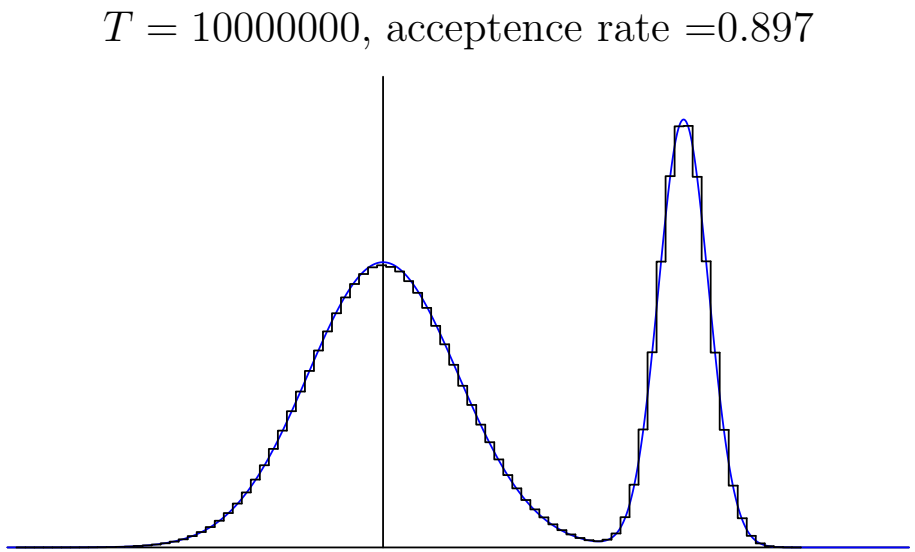
- The transformation method only works when we can easily compute the inverse *cumulative distribution function* (CDF)
- The **rejection method** draws Y from a density g_Y with $cg_Y(x) \geq f_X(x)$ for all x, and accepts it with probability $f_X(Y)/(cg_Y(Y))$
- A deviate is accepted with probability $1/c$, so on average we need c tries for each deviate
- Very general, but for complicated or multivariate distributions it is hard to find a good g_Y, and c can be enormous (it typically grows exponentially with the dimension)

Drawing Normal Deviates



Outline

1. Monte Carlo
2. **MCMC**
3. Modern MCMC



Detailed Balance

- Suppose we have a set of states $\mathcal{S}$ and want to draw samples from a probability distribution $\boldsymbol{\pi} = (\pi_i | i \in \mathcal{S})$
- We invent a dynamical system with a transition probability M_{ij} from state j to state i such that

$$M_{ij}\pi_j = M_{ji}\pi_i$$

- This is known as **detailed balance**
- Summing both sides over j (using $\sum_j M_{ji} = 1$: from i we must go somewhere)

$$\sum_j M_{ij}\pi_j = \sum_j M_{ji}\pi_i = \pi_i \qquad\qquad \mathbf{M}\boldsymbol{\pi} = \boldsymbol{\pi}$$

Convergence of MCMC

- Start from a probability vector $\boldsymbol{p}(0) = \sum_i c_i \boldsymbol{v}^{(i)}$, where $\boldsymbol{v}^{(i)}$ are eigenvectors of $\mathbf{M}$ with eigenvalues λ_i
- If I apply $\mathbf{M}$ many times then

$$\boldsymbol{p}(t) = \mathbf{M}^t \boldsymbol{p}(0) = \mathbf{M}^t \sum_i c_i \boldsymbol{v}^{(i)} = \sum_i \lambda_i^t c_i \boldsymbol{v}^{(i)}$$

- As $t \to \infty$ only the eigenvector(s) with the largest $|\lambda_i|$ survive
- Now $\|\mathbf{M}\boldsymbol{v}\|_1 \leq \|\mathbf{M}\|_1 \|\boldsymbol{v}\|_1 = \|\boldsymbol{v}\|_1$ so $|\lambda_i| \leq 1$, and $\lambda = 1$ has eigenvector $\boldsymbol{\pi}$ ($\mathbf{M}$ is known as a **stochastic matrix**)

(Detailed balance makes $\mathbf{M}$ diagonalisable; if every state can be reached and the chain is not periodic, all other $|\lambda_i| < 1$, so $\boldsymbol{p}(t) \to \boldsymbol{\pi}$)

Metropolis Algorithm

- A very easy way to achieve detailed balance: from state j propose a "neighbouring" state i, each neighbour with the same probability q
- Accept the move if $\pi_i \geq \pi_j$; otherwise accept it with probability π_i/π_j
- If $\pi_i \geq \pi_j$ then $M_{ij} = q$ and $M_{ji} = q\pi_j/\pi_i$. Thus

$$M_{ij}\pi_j = q\pi_j \qquad\qquad M_{ji}\pi_i = q\frac{\pi_j}{\pi_i}\pi_i = q\pi_j$$

- Note that we require the state i to have the same number of neighbours as state j so that detailed balance is satisfied

Continuous Variables and Metropolis-Hastings

- For continuous $\boldsymbol{\theta}$, detailed balance for the transition density W is

$$W(\boldsymbol{\theta} \to \boldsymbol{\theta}')\pi(\boldsymbol{\theta}) = W(\boldsymbol{\theta}' \to \boldsymbol{\theta})\pi(\boldsymbol{\theta}')$$

- Propose $\boldsymbol{\theta}'$ from a **proposal distribution** $p(\boldsymbol{\theta}'|\boldsymbol{\theta})$ and accept with probability

$$\min\left(1, \frac{p(\boldsymbol{\theta}|\boldsymbol{\theta}')\pi(\boldsymbol{\theta}')}{p(\boldsymbol{\theta}'|\boldsymbol{\theta})\pi(\boldsymbol{\theta})}\right)$$

- For a symmetric proposal (e.g. $\boldsymbol{\theta}' = \boldsymbol{\theta} + s\boldsymbol{\epsilon}$) the p's cancel: the Metropolis algorithm

- Otherwise (e.g. when $\theta_i \geq 0$) we need the correction: **Metropolis-Hastings**

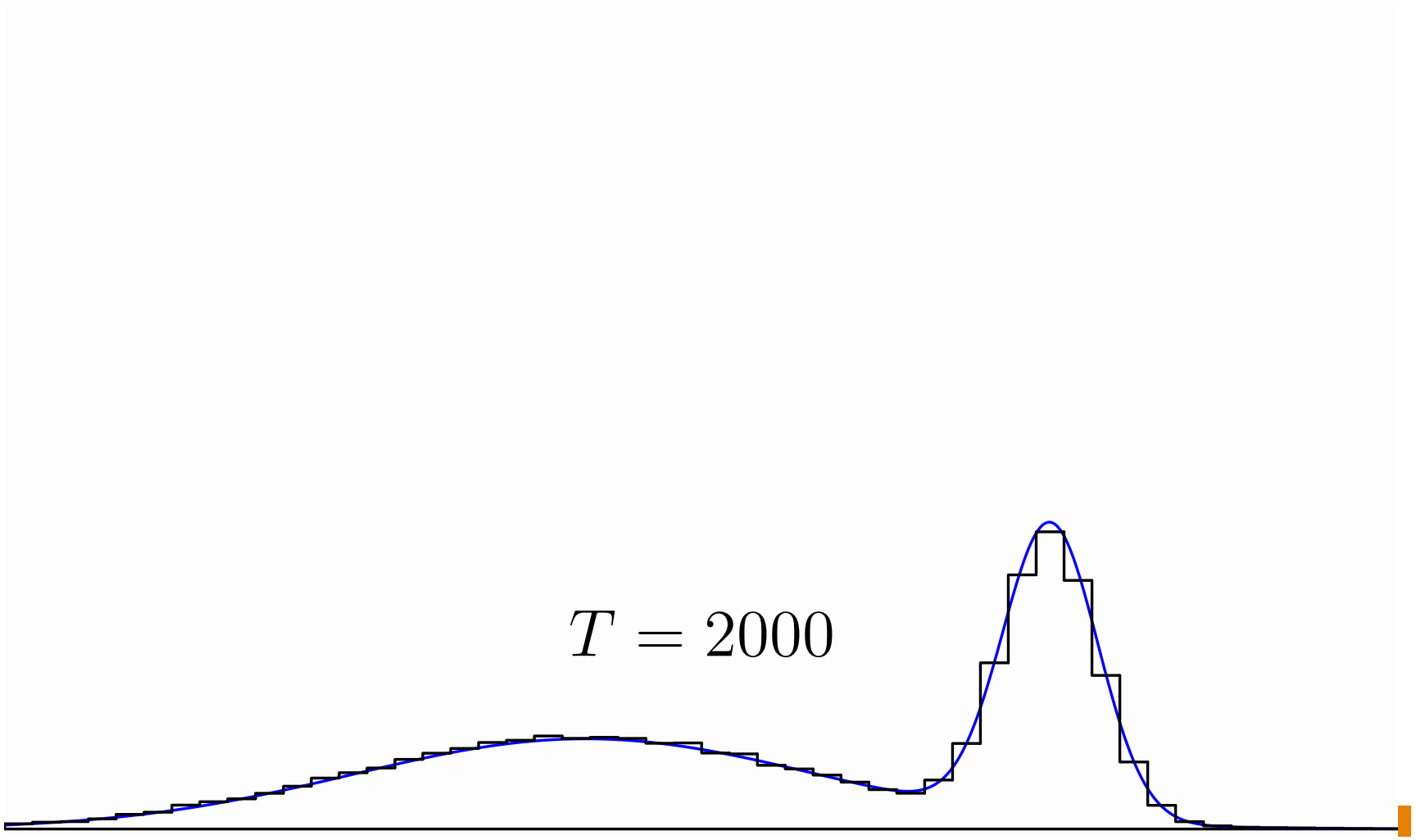
What Makes MCMC Nice

- Because we are free to choose where we move (and choose close by neighbours) $\pi(\boldsymbol{\theta}') \approx \pi(\boldsymbol{\theta})$ so that moves are not too infrequent (but tiny steps explore slowly: the step size is a trade-off)
- Also very importantly the updates depend only on the ratio $\pi(\boldsymbol{\theta}')/\pi(\boldsymbol{\theta})$
- We only need to know our probabilities up to a multiplicative scaling factor
- For sampling from the posterior we only need to know the likelihood and prior $\mathbb{P}(\mathcal{D}|\boldsymbol{\theta})\mathbb{P}(\boldsymbol{\theta})$ (or $f(\mathcal{D}|\boldsymbol{\theta})\,f(\boldsymbol{\theta})$)
- We don't need to know $\mathbb{P}(\mathcal{D})$ which we generally don't know

What Makes MCMC Nasty

- It can take a long time until our states occur with the probability π (i.e. we have forgotten our initial state)
- We don't even know how long we have to wait
- Even when we have reached this *equilibration time* each sample is correlated with the previous sample
- To get a good approximation to the posterior expectation requires running for many times the equilibration time
- Note, if we are just finding sample averages then we can use all samples after equilibrating even if they are not independent

Burn-In



Gibbs Sampling

- With many parameters we can update one component, θ_k, at a time, keeping the others, $\boldsymbol{\theta}_{-k}$, fixed
- **Gibbs sampling** draws the new θ'_k from its conditional distribution $\pi(\theta'_k|\boldsymbol{\theta}_{-k})$
- This is Metropolis-Hastings with proposal $p(\boldsymbol{\theta}'|\boldsymbol{\theta}) = \pi(\theta'_k|\boldsymbol{\theta}_{-k})$, and the acceptance probability is always 1
- Useful when the conditionals are standard distributions (often thanks to conjugate priors), even though the joint posterior is not
- When parameters are strongly correlated, moving one at a time gives small steps and the chain mixes slowly

Traffic Rate

- Consider monitoring the flow of traffic where we have data

$$\mathcal{D} = (N_1, N_2, \ldots, N_n)$$

  where N_i is the number of cars that passed on day i

- We assume $N_i \sim \mathrm{Poi}(\mu)$ and want to infer μ

- The Poisson distribution has a gamma conjugate prior

- We don't have any prior knowledge on μ so we use a non-informative prior $\mathrm{Gam}(\mu|0,0) \propto 1/\mu$ (improper: it does not normalise)

- Note that we can solve this problem exactly—however, let's compare with MCMC

Proposal Distribution

- If we can choose our proposal distribution $p(\mu'|\mu)$ to be close to the posterior distribution then our acceptance rate would be close to 1.

- We choose $p(\mu'|\mu) = \mathrm{Gam}(\mu'|\mu^2,\mu)$ which has $\mathbb{E}[\mu'] = \mu$ and variance 1.

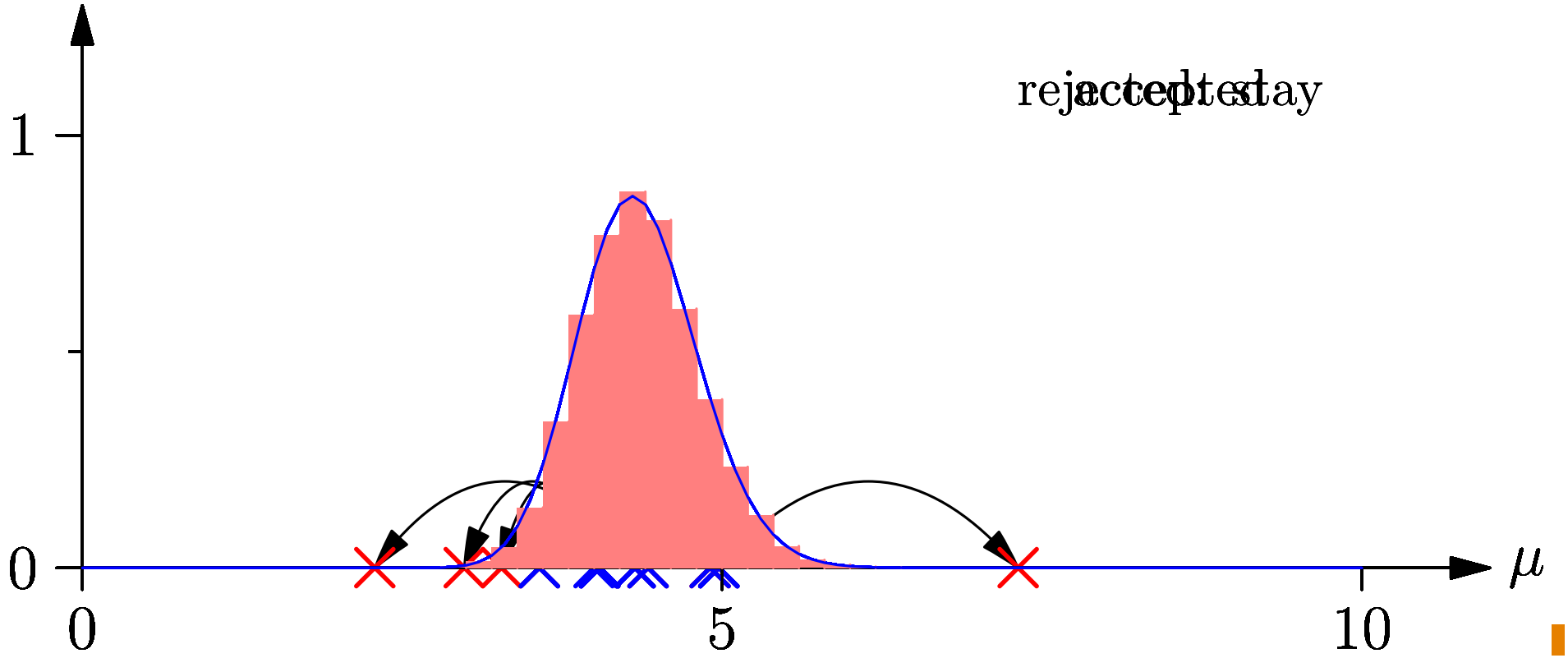
- We update with probability $\min(1,r)$ where

$$r = \frac{\mathrm{Gam}(\mu|\mu'^2,\mu')\frac{1}{\mu'}\prod_{i=1}^n \mathrm{Poi}(N_i|\mu')}{\mathrm{Gam}(\mu'|\mu^2,\mu)\frac{1}{\mu}\prod_{i=1}^n \mathrm{Poi}(N_i|\mu)}$$

$$= \frac{\mu\mathrm{Gam}(\mu|\mu'^2,\mu')}{\mu'\mathrm{Gam}(\mu'|\mu^2,\mu)}\mathrm{e}^{-n(\mu'-\mu)+\sum_{i=1}^n N_i \log\left(\frac{\mu'}{\mu}\right)}.$$

MCMC in Practice

$$\mathcal{D} = \{6, 2, 6, 2, 3, 4, 3, 5, 4, 8, 1, 3, 3, 8, 5, 9, 4, 6, 5, 0\}$$

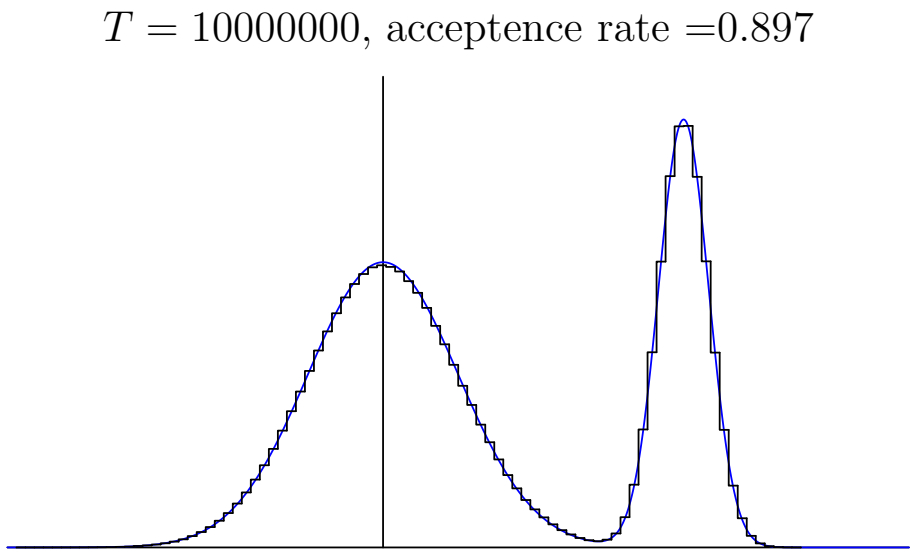


Checking Convergence

- Run several chains from widely different starting points and discard each chain's burn-in
- Plot the traces: after burn-in the chains should overlap and look alike
- $\hat{R}$ compares the variance between chains with the variance within them: close to 1 (say below 1.01) when they agree
- Successive samples are correlated: the **effective sample size** is the number of independent samples they are worth
- Count a state again whenever a move is rejected; libraries do all of this for you

Outline

1. Monte Carlo
2. MCMC
3. **Modern MCMC**



Random Walks Are Slow

- In traditional MCMC choosing a random proposal makes the search into something close to a random walk
- This is often acceptable for a few hundred parameters
- In high dimensional spaces with long shallow valleys it can take an a hugely long time to navigate the high probability landscape
- There is one commonly used trick to speed things up. . .

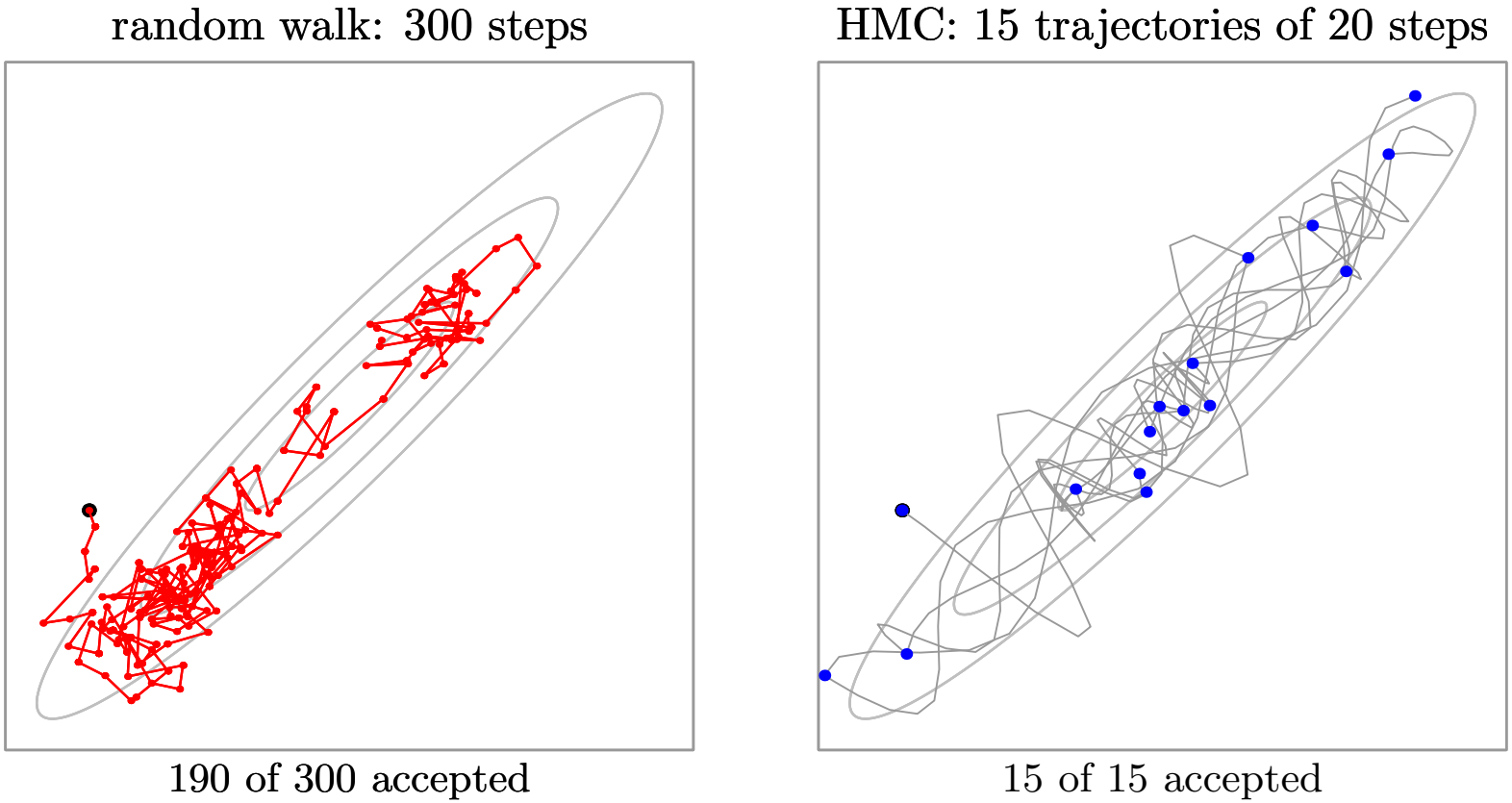
Hamiltonian Monte Carlo

- Treat $U(\boldsymbol{\theta}) = -\log \pi(\boldsymbol{\theta})$ as a landscape (it needs only likelihood $\times$ prior)
- Give the sampler a random momentum $\boldsymbol{p} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ and let it roll without friction, conserving the energy

$$H(\boldsymbol{\theta}, \boldsymbol{p}) = U(\boldsymbol{\theta}) + \tfrac{1}{2}\|\boldsymbol{p}\|^2$$

- It glides along the contours and travels a long way
- We simulate the motion in small steps, which is slightly inexact, so accept the end point with probability $\min(1, \mathrm{e}^{H_{\text{start}} - H_{\text{end}}})$: a Metropolis-Hastings step, so the samples are exact
- H hardly changes, so distant moves are almost always accepted

Comparison of MCMC with HMC



Same start and the same work (300 evaluations) on a strongly correlated Gaussian. The random walk crawls along the ridge; HMC crosses it in every trajectory.

HMC in Practice

- HMC needs the gradient $\nabla \log \pi(\boldsymbol{\theta})$: automatic differentiation (PyTorch, JAX) gives it for free
- Like momentum in optimisation, the gradient says where to go
- It needs a step size and a number of steps: too few and it is a random walk, too many and it wastes time coming back
- The **No-U-Turn Sampler** (NUTS, Hoffman and Gelman 2014) tunes these, stopping when the path's ends stop moving apart
- NUTS is the default in Stan, PyMC and NumPyro: write down the generative story and get samples

The Wider Landscape

- **Variational inference**: fit an approximate posterior by optimisation (the ELBO of the probabilistic inference lecture) —fast, but approximate
- **Stochastic-gradient MCMC**: noisy gradient steps on minibatches, for very large data sets and Bayesian deep learning
- **Parallel tempering** and **sequential Monte Carlo**: for posteriors with several well-separated peaks
- Diffusion models in generative AI draw samples with Langevin-type dynamics, a close relative of MCMC

Conclusions

- As soon as we use complex models we can no longer compute the posterior in closed form
- MCMC samples from the posterior using only likelihood $\times$ prior
- Simple Metropolis-Hastings is a few lines of code, and on modern computers fast for problems with a modest number of parameters
- For larger problems use HMC (NUTS) through a probabilistic programming language
- Always check convergence: several chains, $\hat{R}$, effective sample size